

5 Jazyk SQL (specializace WDOP)

Uvažujte doménu hudební scény.

1. Napište v jazyce SQL příkazy, které vytvoří tabulky pro reprezentaci zpěváků, kapel a skladeb. Vytvořte alespoň dva atributy pro každou z těchto entit (např. jméno a země původu zpěváka) a použijte vhodné datové typy.
2. Upravte vhodnými SQL příkazy tabulky tak, aby bylo možné ukládat informace o členství zpěváků v kapelách a o interpretech skladeb. Předpokládejte, že každý zpěvák je členem maximálně jedné kapely a každá skladba může mít maximálně jednoho interpreta. Kapela ale může mít více zpěváků a zpěvák může interpretovat více skladeb. Zajistěte referenční integritu.
3. Napište SQL příkaz, který vloží do tabulek zpěváka, který je členem nějaké kapely a interpretuje nějakou skladbu.
4. Napište v SQL dotaz, který vypíše jména všech zpěváků, kteří nejsou členy žádné kapely.
5. Vysvětlete, co dělá následující SQL dotaz (obecně i nad aktuálně vloženými daty):

```
SELECT nazev, COUNT(*)
FROM kapela
GROUP BY nazev
HAVING COUNT(*) > 2;
```

6. Napište SQL příkazy, které smažou vytvořené tabulky.

Nástin řešení 1.

```
CREATE TABLE zpevak (id_z INT PRIMARY KEY, jmeno VARCHAR(100), zeme VARCHAR(200));
CREATE TABLE kapela (id_k INT PRIMARY KEY, nazev VARCHAR(100), zalozena INT);
CREATE TABLE skladba (id_s INT PRIMARY KEY, titul VARCHAR(100), delka TIME);
```

2.

```
ALTER TABLE zpevak ADD COLUMN clen INT;
ALTER TABLE zpevak ADD FOREIGN KEY (clen) REFERENCES kapela (id_k);
ALTER TABLE skladba ADD COLUMN interpret INT;
ALTER TABLE skladba ADD FOREIGN KEY (interpret) REFERENCES zpevak (id_z);
```

3.

```
INSERT INTO kapela VALUES (1, 'Queen', 1970);
INSERT INTO zpevak VALUES (101, 'Freddie Mercury', 'Zanzibar', 1);
INSERT INTO skladba VALUES (201, 'Under Pressure', '00:04:08', 101);
```

(Pozor na pořadí nebo ADD FOREIGN KEY až po vložení všech dat.)

4.

```
SELECT jmeno FROM zpevak z
WHERE NOT EXISTS(SELECT * FROM skladba WHERE interpret = z.id_z);
```

5. Vrací název a počet členů kapel, které mají více než dva zpěváky. Pro daná data nevrátí nic.

6.

```
DROP TABLE skladba;
DROP TABLE zpevak;
DROP TABLE kapela;
```

(Pozor na pořadí nebo nejdřív DROP CONSTRAINT.)

6 Moderní databázové systémy (specializace WDOP)

Uvažujte problematiku multi-modelových dat a databází.

1. Vysvětlíte pojem „multi-modelová data“ a rozdíl mezi multi-model a single-model databázemi. Uveďte příklad.
2. Uveďte alespoň dvě výhody a dvě nevýhody multi-modelových databází oproti tradičním single-model databázím.
3. Pro libovolnou multi-modelovou databázi vytvořte jednoduchý příklad databázového schématu, resp. dat a nad nimi multi-modelový dotaz. Demonstrujte na něm uvedené výhody multi-modelových databází.
4. Vysvětlíte rozdíl mezi multi-modelovou databází a polystorem.

Nástin řešení 1. Multi-modelová data jsou data, která kombinují více než jeden model. Multi-modelové databáze jsou databáze, které takovou kombinaci nativně podporují. Např. PostgreSQL kombinuje relační a dokumentový model pomocí embeddingu, tj. sloupec relační tabulky může být typu JSON/JSONB a obsahovat dokumenty. Single-model databáze podporují jen jeden model.

2. Výhody: data jsou uložena nativně pomocí optimálního modelu, jsou uložena v jediném systému (ne v několika single-model systémech), existuje k nim jediný společný dotazovací jazyk, ...

Nevýhody: neexistuje žádný standard pro kombinaci modelů (jaké, jak), standardní dotazovací jazyk (kromě SQL/XML a SQL/JSON pro relační/dokumentová data) apod., implementace multi-model systému je složitá (modely mají z principu protichůdné vlastnosti), existující systémy mají stále mnoho omezení, ...

3. Např. v PostgreSQL vytvoříme klasickou relační tabulku se sloupcem pro dokumentová data a dotážeme se na ně:

```
CREATE TABLE zakaznik (
  id          INTEGER PRIMARY KEY,
  jmeno       VARCHAR(50),
  objednavky  JSONB
);
INSERT INTO zakaznik
VALUES (1, 'Karel Novák',
'{"Cislo_obj": "o43",
  "Polozky": [
    {"Id_produkту": "p34", "Jmeno_produkту": "Myš", "Cena": 230, "Pocet": 2},
    {"Id_produkту": "p56", "Jmeno_produkту": "Klávesnice", "Cena": 788}
  ]}');
INSERT INTO zakaznik
VALUES (2, 'Petr Novák',
'{"Cislo_obj": "o33",
  "Polozky": [
    { "Id_produkту": "p777", "Jmeno_produkту": "Počítač", "Cena": 34000 }
  ]}');

SELECT jmeno,
  objednavky->>'Cislo_obj' as cislo_obj,
  objednavky#>'Polozky,0'>>'Jmeno_produkту' as Jmeno_produkту
FROM zakaznik
WHERE objednavky->>'Cislo_obj' <> 'o33';
```

Jak je vidět, tak nám stačí jeden systém, ale rozšíření jazyka SQL o konstrukty pro JSON data je systémově specifické, další modely nemusíme mít podporované atd.

4. Polystore stejně jako multi-model databáze ukládá multi-modelová data, ale pro jednotlivé modely využívá samostatné single-model databáze. Centrální prvek (mediator) pak zajišťuje jejich management, dekompozici a vyhodnocení dotazu, spojení mezivýsledků atd. Hlavní výhodou je využití robustních prověřených single-model systémů, nevýhodou je komplexita mediatoru, malá podpora v komerčním světě (spíš akademické systémy).

7 Web (specializace WDOP)

1. Vysvětlíte návrhový vzor Front Controller a jeho využití pro vývoj webových aplikací. S pomocí pseudokódu nebo PHP demonstřujete základní myšlenku implementace.

2. Uvažujte následující webové (REST) API:

– GET /api/singer – vrátí JSON, který je polem identifikátorů zpěváků, například:

```
["jaroslav-1980", "pavelka-1958"]
```

– GET /api/singer/{identifikátor} – vrátí JSON, který obsahuje jméno zpěváka, například:

```
{ "name": "Jaroslav" }
```

– DELETE /api/singer/{identifikátor} – smaže zpěváka

Pro popsané API napište v JavaScriptu, který poběží ve webovém prohlížeči, funkci, která dostane jako argument jméno zpěváka a provede následující kroky:

1. smaže všechny zpěváky daného jména,

2. pro každého smazaného zpěváka přidá HTML element li se jménem zpěváka do elementu s id `deleted-list`.

Pokud si nejste jisti názvem nějaké funkce/proměnné, popište její chování. Výsledný kód může být strukturován do více funkcí, na drobné syntaktické chyby nebude při hodnocení brán zřetel. V kódu nemusíte řešit obsluhu chybových stavů.

Nástin řešení 1. Základní myšlenkou je mít jedno vstupní místo do aplikace pro obsluhu požadavků od klienta. Návrhový vzor je použitý na straně serveru.

V zásadě stačí switch, nebo třída podobného stylu. Skládá ze dvou částí – routing a dispatching, tedy například:

```
// Routing.
$handler = routing($_GET, $_SERVER);
// Necháme metodu obsloužit - dispatching.
$handler->handle($_GET, $_POST, $_SERVER);
```

Dále je možné uvést například inicializaci a ošetření chybového stavu.

2. Implementace by měla obsahovat fetch na seznam a pak na jednotlivé zpěváky. Následně fetch, který provede delete. Poslední je vytvoření a vložení HTML elementu.

Například:

```
async function deleteSingersByName(name) {
  const identifiers = await fetchJson("./api/singer");
  for (const identifier of identifiers) {
    const url = "./api/singer/" + encodeURIComponent(identifier);
    const singer = await fetchJson(url);
    if (singer.name !== name) {
      continue;
    }
    await fetch(url, {method: "DELETE"});
    addToList(singer.name);
  }
}
```

```

async function fetchJson(url) {
    return await (await fetch(url)).json();
}

function addToList(name) {
    const li = document.createElement("li");
    li.innerText = name;
    document.getElementById("deleted-list").appendChild(li);
}

```

8 Základy indexování (specializace WDOP)

Pro potřeby této otázky uvažujme databázi vytvořenou v první otázce specializace („Jazyk SQL”).

1. Vysvětlíte přímé/nepřímé indexování a primární/sekundární index.
2. Uveďte příklady možného použití přímého primárního, přímého sekundárního a nepřímého sekundárního indexu pro tabulku reprezentující zpěváky (zpevak). Jaké jsou výhody a nevýhody použití indexů?
3. Předpokládejme přímé primární indexování sloupce zpevak.id (identifikátor zpěváka) pomocí redundantního B-stromu bez odloženého štěpení, stupně 3, tedy 2 hodnoty a 3 ukazatele. Demonstrujte postupné vkládání záznamů s hodnotou klíče: 1, 2, 3, 4, 5, 0.

Nástin řešení 1. Přímé indexování ukazuje přímo na data, nepřímé indexování ukazuje na jiný index. Primární index odpovídá fyzickému pořadí uložení záznamů, sekundární index mu neodpovídá.

2. Přímý primární – primární klíč, zpevak.id_z. Takový klíč může být jen jeden. Sekundární indexy je možné použít na všechny ostatní sloupce, kde očekáváme časté dotazování. Přímá a nepřímá varianta je zde jen implementačním detailem.

Výhodou indexu je rychlejší dotazování, nevýhodou pak pomalejší vkládání a mazání, kdy je třeba index upravit.

3. Postupně, první 4 jsou zapsány na jeden řádek, jedná se vždy o kořen a jeho potomky. Úrovně jsou oddělené čárkou.

1:

[1]

2:

[1,2]

3: Tady je možné [2], [1] [2,3] i [2], [1,2] [3] – záleží na tom, na jakou stranu uvažujeme ostrou nerovnost, jestli '<,>=' ,nebo '<=,>'. Důležité je, že je třeba být v tomto konzistentní i ve zbytku řešení.

4:

'<,>=': [2,3], [1] [2] [3,4]

'<=,>': [2], [1,2] [3,4]

5:

'<,>=': [3], [2] [4], [1] [2] [3] [4,5]

'<=,>': [2,4], [1,2] [3,4] [5]

0:

'<,>=': [3], [2] [4], [0,1] [2] [3] [4,5]

'<=,>': [2], [1] [4], [0,1] [2] [3,4] [5]